Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 31

Базовые меры безопасности full-stack приложения: защита от XSS, Helmet и CSP

Вариант 2 – лендинг IT-конференции

Выполнила: Клычева Г.

Группа: 310102

Репозиторий: https://github.com/karrinkl/devconf-security.git

Проверил: Прудник А.М.

1 ЦЕЛЬ РАБОТЫ И УСЛОВИЕ ЗАДАНИЯ

Цель работы состоит в получении практических навыков защиты веб-приложения от межсайтового выполнения сценариев (XSS). Требуется найти место, где пользовательский ввод попадает на страницу, показать атаку, затем закрыть уязвимость очисткой данных на сервере, заголовками Helmet и политикой безопасности контента (CSP) и повторить атаку.

Для варианта 2, лендинга IT-конференции DEVCONF 2026, вектором атаки выбрано поле «Вопрос спикеру». Условие работы включает пять задач:

– найти место, где ввод пользователя попадает на страницу, и показать XSS до защиты;

– очищать поля формы на сервере до записи в базу;

– подключить Helmet.js и базовую CSP со значением script-src 'self';

– повторить атаку и показать, что скрипт не выполняется;

– показать заголовки ответа сервера.

Работа продолжает API из лабораторной работы № 28 (Express, MongoDB, сущность «Спикер») и выполнена в репозитории devconf-security. Страницу с вопросами обслуживает тот же сервер Express, поэтому заголовки Helmet и CSP относятся именно к той странице, на которой вопрос виден. Клиентский лендинг на Next.js из лабораторной работы № 30 в этой работе не менялся.

2 ГДЕ ВВОД ПОПАДАЕТ НА СТРАНИЦУ

Форма на странице /questions принимает три поля: имя автора, имя спикера и текст вопроса. Сервер сохраняет их в коллекцию questions в MongoDB и сразу же рисует список заданных вопросов в HTML. Модель

Question ограничивает только длину строк: от 2 до 80 символов для имени, от 2 до 120 для имени спикера и от 5 до 1000 для текста вопроса.

Опасность возникает в тот момент, когда сохранённая строка подставляется в разметку без кодирования. Если автор вопроса отправит текст с HTML-тегом, браузер каждого посетителя разберёт его как часть страницы и выполнит. Такая атака называется хранимой XSS: нагрузка лежит в базе и срабатывает у всех, кто открыл список вопросов, а не только у самого автора.

Для демонстрации в проекте предусмотрен режим SECURITY=off, который запускается командой npm run demo:unsafe. В этом режиме сервер записывает строку в базу как есть, вставляет её в HTML без кодирования и не отправляет заголовок CSP. Политика CSP снята намеренно: с ней браузер заблокировал бы скрипт, и показать уязвимость было бы невозможно. Обычный запуск npm run local этот режим не включает, а на странице в нём показана жёлтая плашка с предупреждением.

3 АТАКА ДО ЗАЩИТЫ

В поле «Вопрос» отправлена такая нагрузка:

```
<img src=x onerror="this.outerHTML='<strong>XSS выполнен</strong>'">
```

Классическая нагрузка из методических материалов, <script>alert('XSS PoC')</script>, в том же режиме тоже попала бы в исходный документ и выполнилась: тег script в разметке страницы браузер исполняет. Вариант с onerror выбран потому, что он не зависит от того, как именно вставлена разметка. Через свойство innerHTML тег script, напротив, не выполняется, поэтому страница в этой работе отрисовывается на сервере, а не вставляется сценарием на клиенте.

Картинки по адресу x не существует, поэтому браузер вызывает обработчик onerror, и на месте тега появляется жирный текст «XSS выполнен». Результат приведён на рисунке 1: текст создал сценарий из обработчика, а не автор формы.

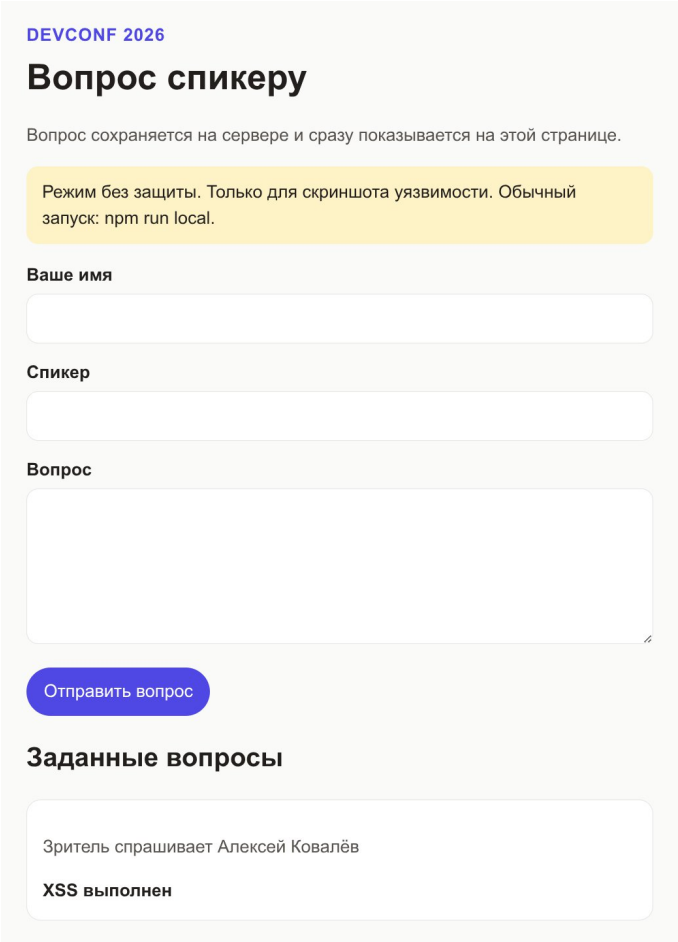


Рисунок 1 – Страница вопросов в режиме без защиты: нагрузка выполнилась

4 ЗАЩИТА

Защита построена в три независимых слоя. Каждый следующий слой нужен на случай, если предыдущий когда-нибудь пропустит разметку.

4.1 Очистка на сервере

Файл lib/sanitize.js содержит функцию sanitizePlainText. Она использует библиотеку sanitize-html: HTML разбирается так же, как в браузере, а не регулярным выражением, которое легко обойти. Список разрешённых тегов пустой, поэтому тег script выбрасывается вместе с кодом внутри, а у остальных тегов снимается разметка и остаётся только текст. Ядро функции:

```
return sanitizeHtml(source, {
  allowedTags: [],
  allowedAttributes: {},
})
  .replace(/\s+/g, ' ')
  .trim();
```

Функцию вызывает readField в controllers/questionController.js для каждого поля формы. Значение сначала проходит очистку, и только после неё проверяется длина. Поэтому нагрузка, состоящая только из тегов, после

очистки оказывается короче допустимого и отклоняется, а в базу не попадает ничего. Строка сообщения об ошибке на листинге перенесена для печати:

```
function readField(body, key) {
  const rule = LIMITS[key];
  const value = sanitizePlainText(body[key]);

  if (value.length < rule.min || value.length > rule.max) {
    return { error: `${rule.label}: от ${rule.min} до ${rule.max}
      символов после очистки` };
  }

  return { value };
}
```

Цепочка вызовов такая: readField вызывается из parseQuestion, а parseQuestion из createQuestion, и только после успешной проверки выполняется Question.create. Очистка происходит до записи в MongoDB, поэтому в базе хранится уже безопасный текст.

4.2 Кодирование при выводе

Второй слой работает на выходе. Функция escapeHtml в том же файле заменяет символы &, <, >, двойную и одинарную кавычку на HTML-сущности, и браузер показывает их как текст. Шаблон views/questionsPage.js в защищённом режиме пропускает через неё каждое поле:

```
function field(question, key) {
  const value = question[key] || '';
  return isProtectionOn() ? escapeHtml(value) : value;
}
```

Скрипт public/questions.js, который после отправки формы заново строит список, не использует innerHTML. Текст карточки задаётся через textContent, и браузер не разбирает его как разметку. Кодирование защищает и те записи, которые уже лежали в базе до включения очистки.

4.3 Helmet и CSP

Третий слой задаётся заголовками ответа. Файл middleware/security.js подключает Helmet и настраивает CSP. В защищённом режиме функция securityHeaders возвращает следующую конфигурацию:

```
return helmet({
  strictTransportSecurity: false,
  crossOriginResourcePolicy: { policy: 'cross-origin' },
  contentSecurityPolicy: {
    useDefaults: false,
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'"],
      imgSrc: ["'self'", 'data:'],
      connectSrc: ["'self'"],
      objectSrc: ["'none'"],
      baseUri: ["'self'"],
      formAction: ["'self'"],
      frameAncestors: ["'none'"],
    },
  },
});
```

Ключевая директива здесь script-src 'self': браузер выполняет только сценарии, загруженные с адреса самого сервера. Собственный скрипт страницы /questions.js и стиль /style.css разрешены, а встроенный сценарий и обработчик onerror запрещены. Подключение выполняется в server.js вызовом app.use(securityHeaders()) до объявления маршрутов, чтобы заголовки получал каждый ответ.

В конфигурации есть два осознанных решения. Заголовок Strict-Transport-Security (HSTS) выключен: стенд открывается по адресу http://localhost, и с этим заголовком браузер запомнил бы, что localhost бывает только по HTTPS, после чего страница перестала бы открываться. Значение Cross-Origin-Resource-Policy оставлено cross-origin, чтобы лендинг с другого порта по-прежнему мог читать /api/speakers.

5 ПОВТОРНАЯ АТАКА

После включения защиты через форму и API отправлены три нагрузки. Результаты сведены в таблицу 1.

Таблица 1 – Проверка защиты

Что отправили	Что получилось
Расскажите про CSP <script>alert('XSS PoC')</script>	В базе и на странице осталось «Расскажите про CSP». Код внутри script удалён, оповещение не появилось. Ответ API: 201
<script>alert('XSS PoC')</script>	Ответ 400, вопрос не сохранён: после очистки строка короче 5 символов
<img src=x onerror=alert(document.cookie)>	Ответ 400: тег выброшен, текста не осталось

Первая нагрузка показывает, что полезная часть вопроса сохраняется, а вредоносная отбрасывается. Две другие целиком состоят из разметки, поэтому очистка оставляет пустую строку, и сервер отвечает кодом 400.

Запись с сырой нагрузкой из раздела 3 осталась в базе от первого запуска. Как её показывает защищённая страница, видно на рисунке 2. Вопрос Марии к Дарье Мельник отображается как «Расскажите про CSP» без тега script, а старая запись выведена обычным текстом: виден сам тег img с обработчиком onerror, жирная надпись «XSS выполнен» не создаётся.

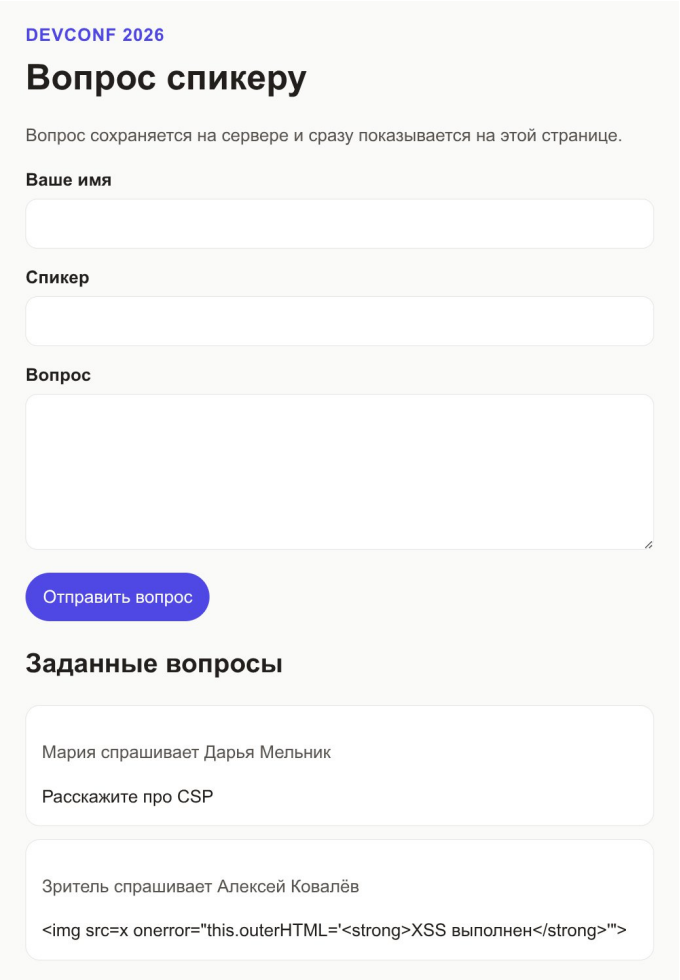


Рисунок 2 – Страница вопросов с включённой защитой

Рисунок подтверждает работу второго слоя: запись, пропущенная в базу до защиты, больше не исполняется, потому что при выводе кодируются угловые скобки и кавычки. Даже если бы разметка прошла очистку и кодирование, обработчик onerror заблокировала бы политика CSP.

6 ЗАГОЛОВКИ ОТВЕТА

Заголовки ответа на запрос GET http://localhost:5000/questions получены командой curl -I. Это вывод командной строки, а не снимок вкладки «Сеть» инструментов разработчика. Основные значения приведены в таблице 2.

Таблица 2 – Заголовки ответа страницы /questions

Заголовок	Значение
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:; connect-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'
X-Content-Type-Options	nosniff
X-Frame-Options	SAMEORIGIN
Referrer-Policy	no-referrer
Cross-Origin-Resource-Policy	cross-origin
Cross-Origin-Opener-Policy	same-origin
X-XSS-Protection	0

Заголовок Content-Security-Policy соответствует настройке из раздела 4.3. Значение nosniff запрещает браузеру угадывать тип содержимого, X-Frame-Options не даёт встроить страницу во фрейм чужого сайта, а Referrer-Policy со значением no-referrer не передаёт адрес страницы при переходах.

Заголовок X-XSS-Protection: 0 выставляет сам Helmet. Он отключает устаревший встроенный фильтр браузера, который сам создавал уязвимости, и не означает, что защита от XSS выключена. Защиту обеспечивают очистка на сервере, кодирование при выводе и CSP. Заголовка Strict-Transport-Security в ответе нет, потому что HSTS выключен намеренно, как описано в разделе 4.3.

7 ЗАПУСК ПРОЕКТА

Проект запускается из каталога devconf-security двумя командами:

```
npm install
npm run local
```

Страница с вопросами открывается по адресу http://localhost:5000/questions, API спикеров доступен по адресу http://localhost:5000/api/speakers. Отдельно устанавливать MongoDB не нужно: команда npm run local поднимает её сама через пакет mongodb-memory-server, а данные хранятся в каталоге .mongo-data. Режим без защиты для повторения скриншота «до» запускается командой npm run demo:unsafe, для обычной работы он не нужен.

ЗАКЛЮЧЕНИЕ

В ходе работы найдена и продемонстрирована хранимая XSS в поле «Вопрос спикеру» лендинга DEVCONF 2026: нагрузка с обработчиком onerror, сохранённая в MongoDB, выполнялась у каждого посетителя страницы. Уязвимость закрыта тремя слоями. Сервер очищает поля формы библиотекой sanitize-html до записи в базу, шаблон страницы кодирует текст при выводе, а

Helmet с политикой CSP script-src 'self' запрещает встроенные и чужие сценарии.

Повторная атака показала, что скрипты из текста вопроса не выполняются, а запись с вредоносной нагрузкой выводится как обычный текст. Заголовки ответа получены командой curl -I и подтверждают настройку CSP. Вопрос больше не исполняется как HTML, а скрипты страницы ограничены собственным источником.

Границы выполненной защиты следует учитывать отдельно. Политика CSP и заголовки Helmet настроены только для сервера Express, который отдаёт страницу вопросов. Клиентский лендинг на Next.js из лабораторной работы № 30 в этой работе не менялся и этих заголовков не получил. Режим SECURITY=off служит только для демонстрации уязвимости и не входит в постоянную защиту. Для боевого развёртывания по HTTPS дополнительно потребуется включить HSTS.